

Customer Segmentation & Clustering with K-Means

Unsupervised segmentation of mall customers using K-Means clustering, built on demographic and spending-behavior data. The project identifies distinct customer groups that can inform targeted marketing, loyalty programs, and personalized promotions.

Table of Contents

- [Project Overview](#)
- [Dataset Summary](#)
- [EDA Using Python](#)
- [Clustering Bivariate \(Annual Income vs Spending Score\)](#)
- [Clustering Bivariate \(Age vs Spending Score\)](#)
- [Key Takeaways & Recommendations](#)
- [Tech Stack](#)

Project Overview

Retailers and mall operators sit on transactional and membership data but often lack a systematic way to translate it into action. This project applies **K-Means clustering**, an unsupervised machine learning technique, to group mall customers into segments based on measurable behavior rather than assumptions.

Business objective: answer the question "*which customers behave alike, and how should we treat each group differently?*"

Approach: 1. Explore the dataset (univariate and bivariate EDA) to understand distributions, gender effects, and relationships between variables. 2. Cluster customers on two bivariate feature pairs — **Annual Income vs Spending Score** and **Age vs Spending Score** — using K-Means. 3. Use the **Elbow Method** (inertia vs. number of clusters) to select the optimal number of clusters for each pairing. 4. Profile each resulting cluster by its average age, income, and spending score to give it a business-friendly label (e.g., "high income, low spender"). 5. Deploy a lightweight **Streamlit app** that takes a customer's income and spending score and predicts which segment they belong to.

Why K-Means: it's fast, scales well to this dataset size (200 rows), and produces clusters that are easy to visualize and explain to non-technical stakeholders — a good fit for a first-pass segmentation exercise.

Dataset Summary

The project uses the classic **Mall Customers dataset** (`Mall_Customers.csv`), 200 records, 5 columns:

Column	Type	Description
<code>CustomerID</code>	int	Row identifier (1-200)
<code>Gender</code>	object	Male / Female
<code>Age</code>	int	Customer age in years
<code>Annual Income (k\$)</code>	int	Yearly income, in thousands of dollars
<code>Spending Score (1-100)</code>	int	A score assigned by the mall based on customer behavior and spending nature

Descriptive statistics:

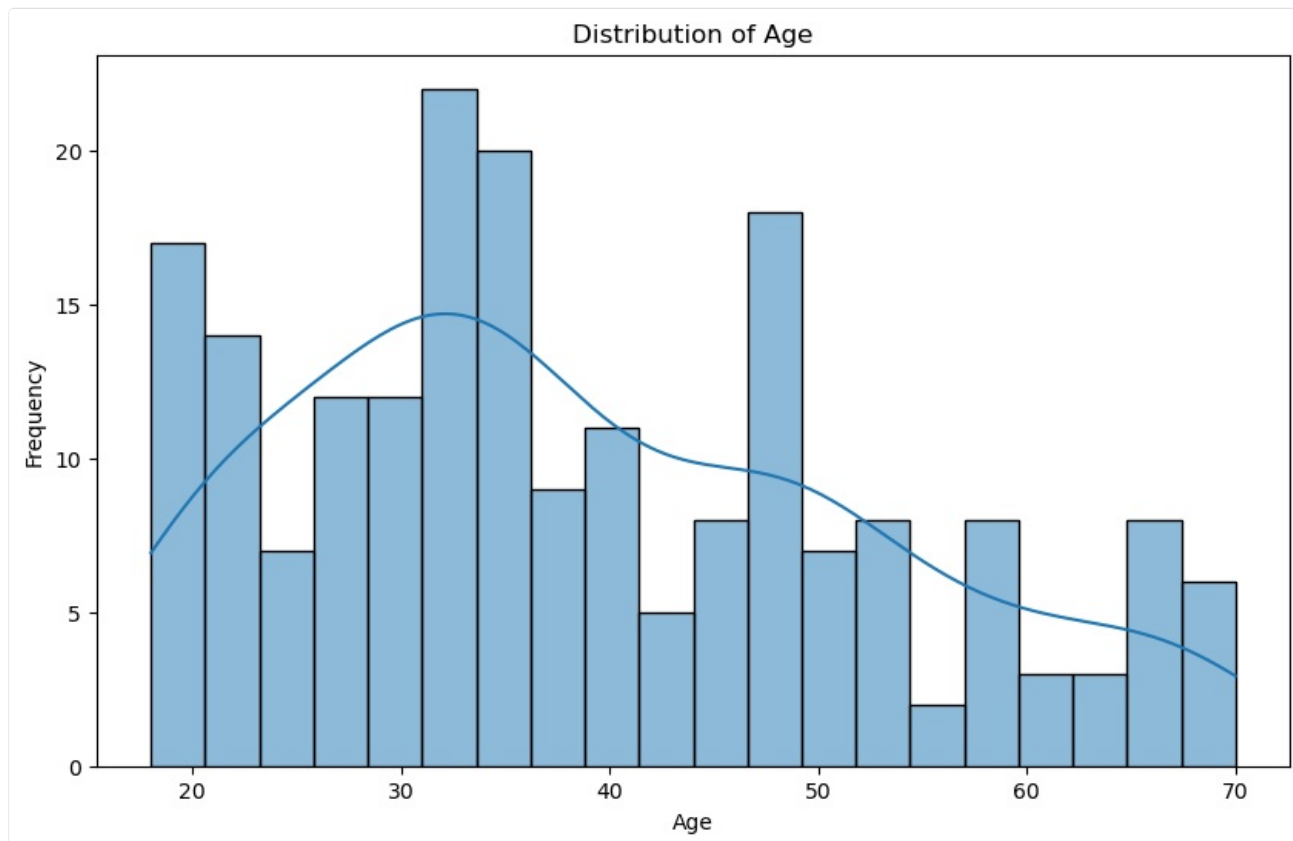
Metric	Age	Annual Income (k\$)	Spending Score (1-100)
Mean	38.85	60.56	50.20
Std Dev	13.97	26.26	25.82
Min	18	15	1
25%	28.75	41.5	34.75
Median	36	61.5	50
75%	49	78	73
Max	70	137	99

No missing values are present across any of the 200 rows (confirmed via `df.info()`), so no imputation was required.

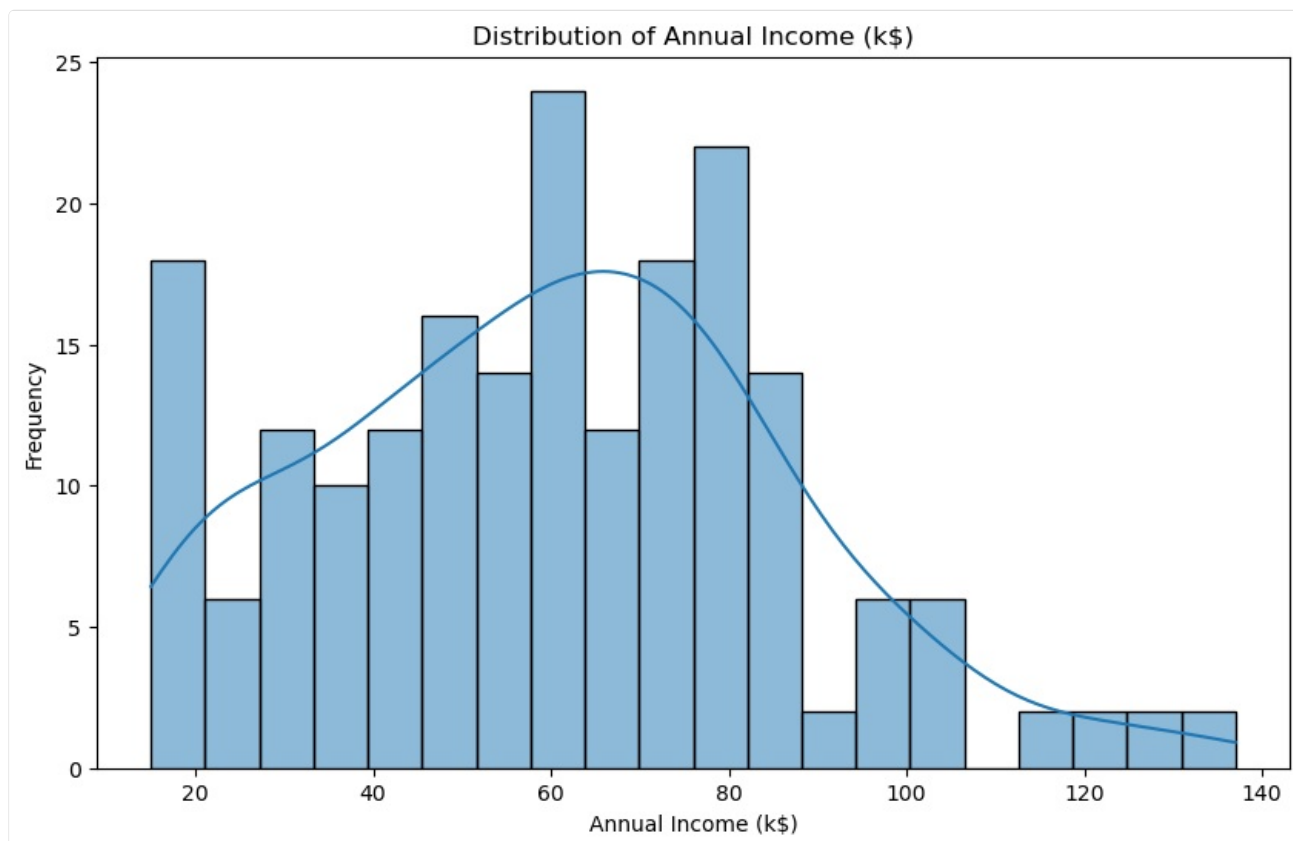
Gender split: 56% Female, 44% Male — a modest skew, not enough to require rebalancing but worth keeping in mind when interpreting gender-based comparisons.

EDA Using Python

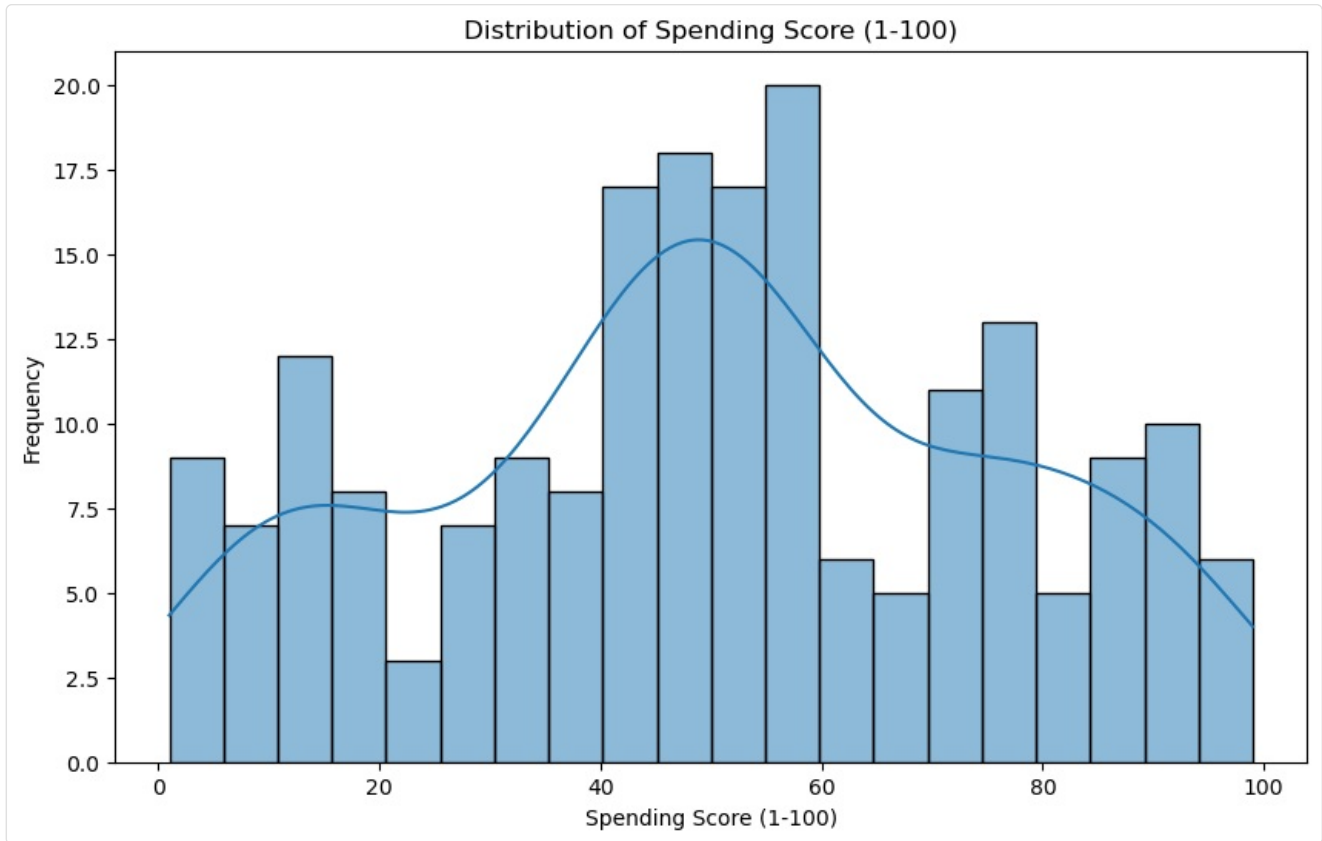
1. Univariate distributions



Age is right-skewed toward younger adults, with a visible concentration in the 25–35 range and a long-ish tail out to 70. There isn't a single dominant "typical" customer age — the mall serves a fairly broad adult age range.

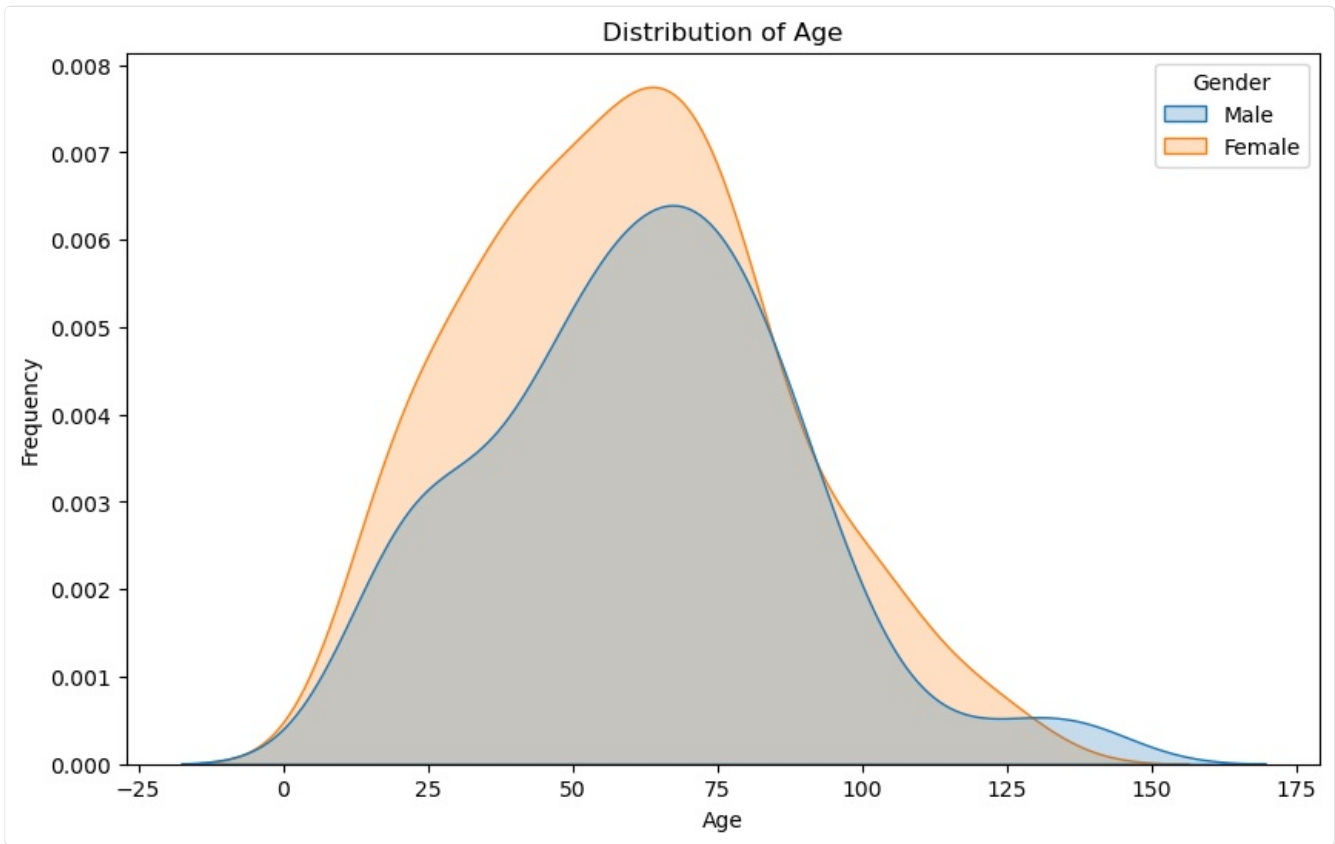


Income is roughly bell-shaped but with a secondary bump around the 70–80k range, hinting that the sample may blend two income sub-populations rather than being a single smooth distribution. Most customers sit between 40k and 80k.

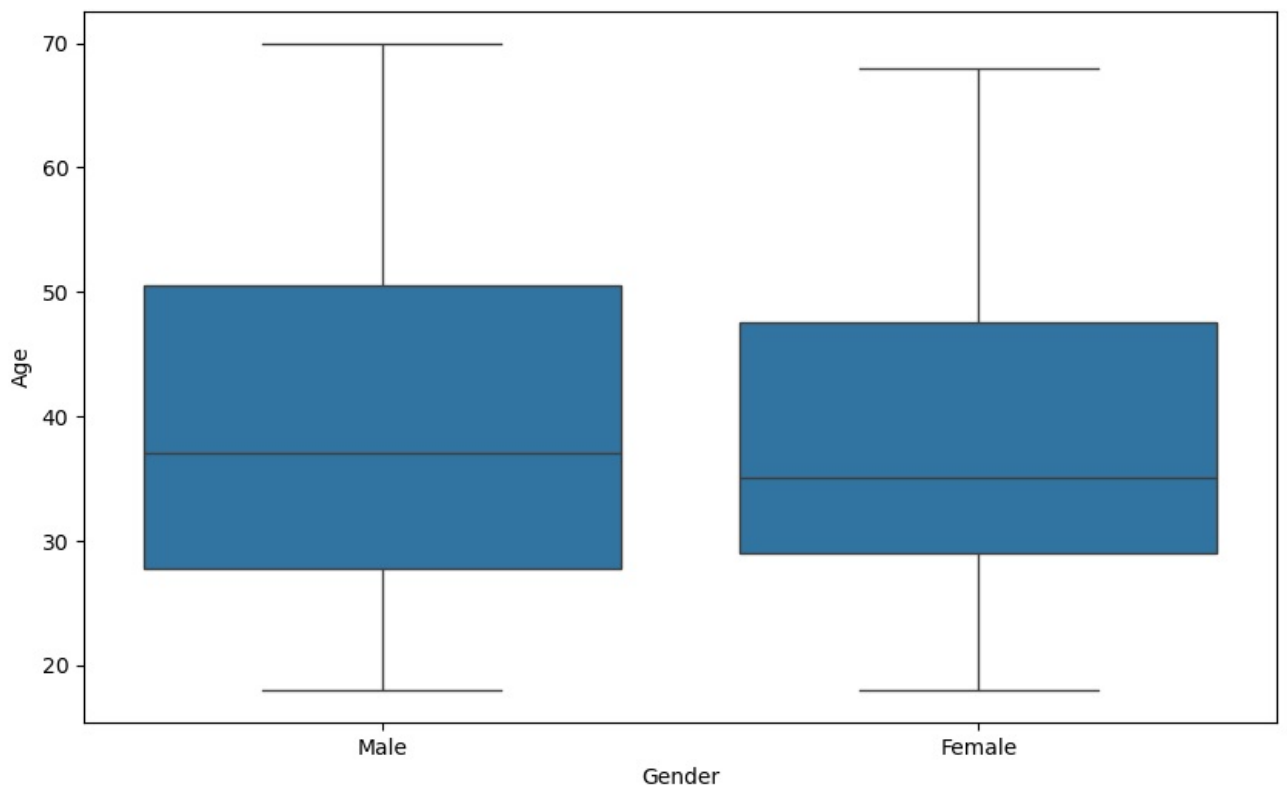


Spending Score is close to uniform across its full 1–100 range, with mild peaks near the middle (~40–60) and at the high end (~70–80). This near-uniformity is actually useful for clustering — it means there's no single "average spender" that dominates, so K-Means has genuine structure to separate.

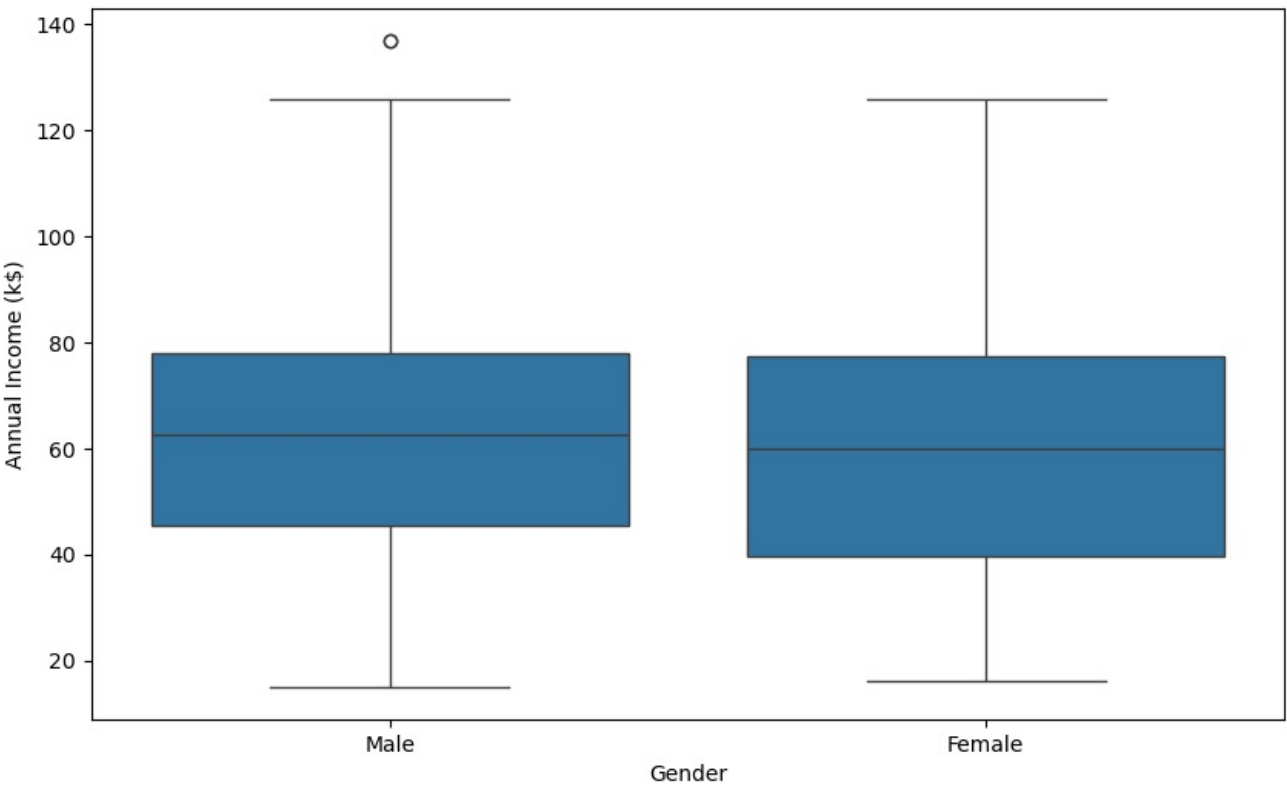
2. Distribution by gender



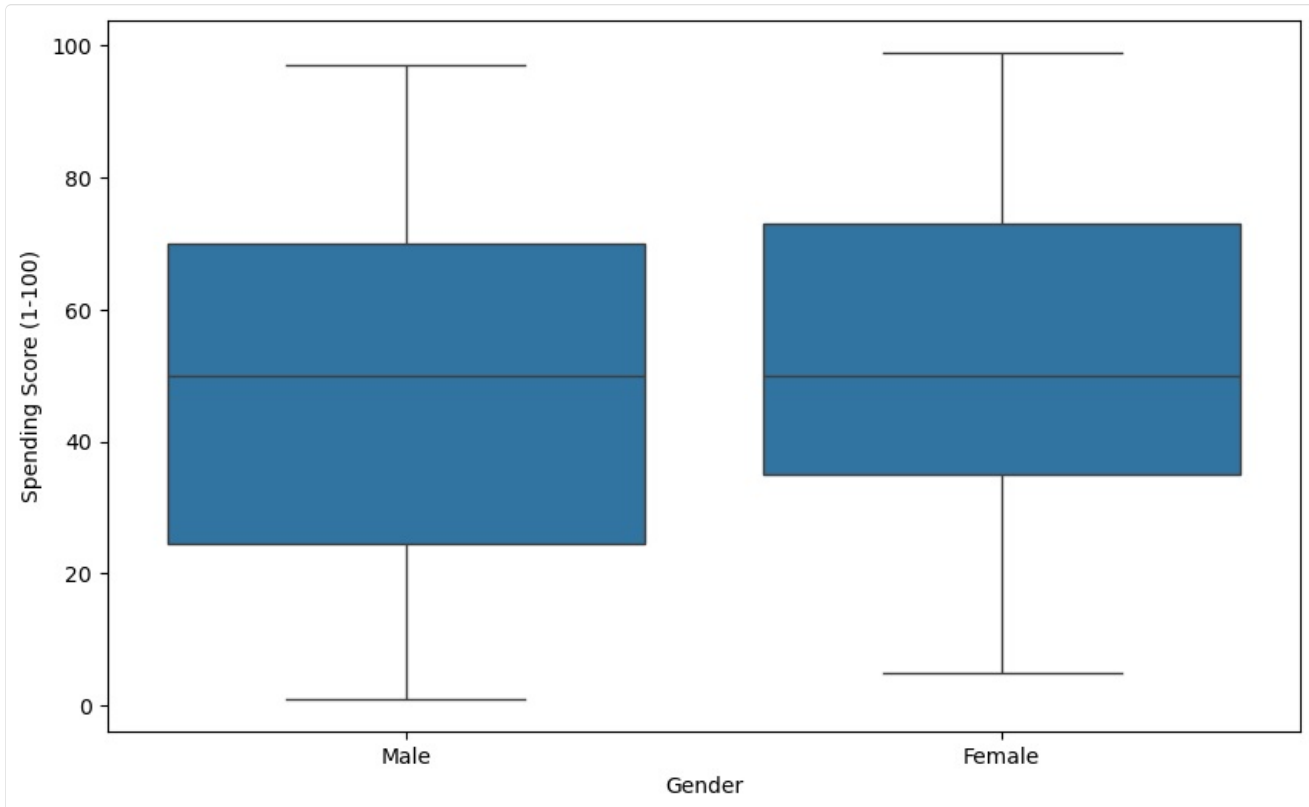
Comparing income density curves for Male vs Female customers shows heavily overlapping distributions — gender is not a strong differentiator of income in this dataset. (Note: the original notebook's loop reused the same `Annual Income` column for all three iterations of this cell rather than switching to `Age` and `Spending Score` in turn — this is a small copy-paste bug worth fixing if you want three genuinely distinct KDE plots. I've only included the one that is correctly labeled.)



Median age is very similar between genders, though Male customers show a slightly wider spread with a higher upper range.



Income medians are close between genders; Male customers have a marginally higher median income (consistent with the groupby means: Female $\approx$ 59.3k vs Male $\approx$ 62.2k) but the boxes overlap substantially, so this isn't a strong effect.

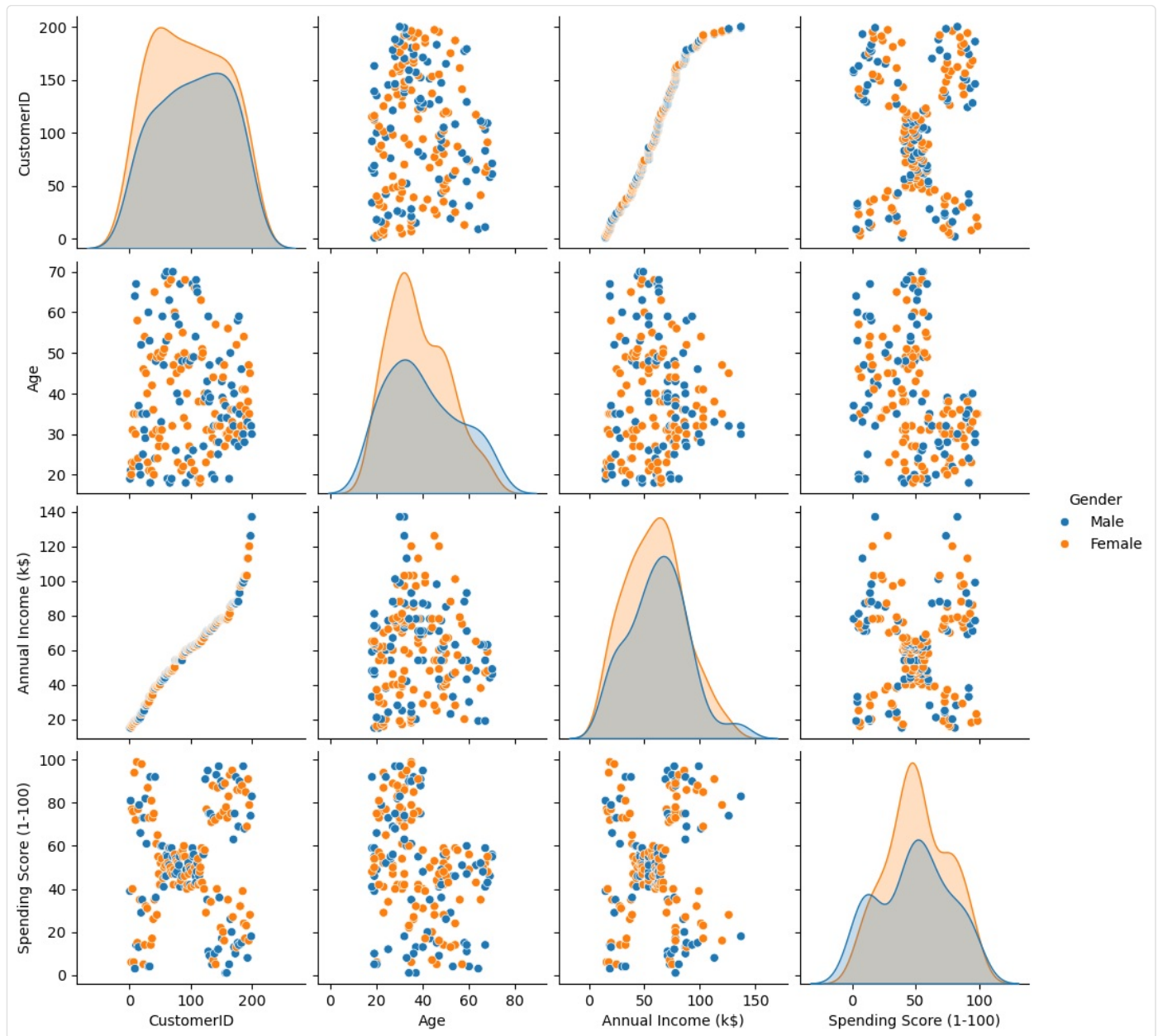


Spending score medians are nearly identical across genders (Female ≈ 51.5 , Male ≈ 48.5). Gender is not a meaningful predictor of spending behavior on its own — this justifies clustering on income/age/spending rather than segmenting simply by gender.

3. Bivariate relationships

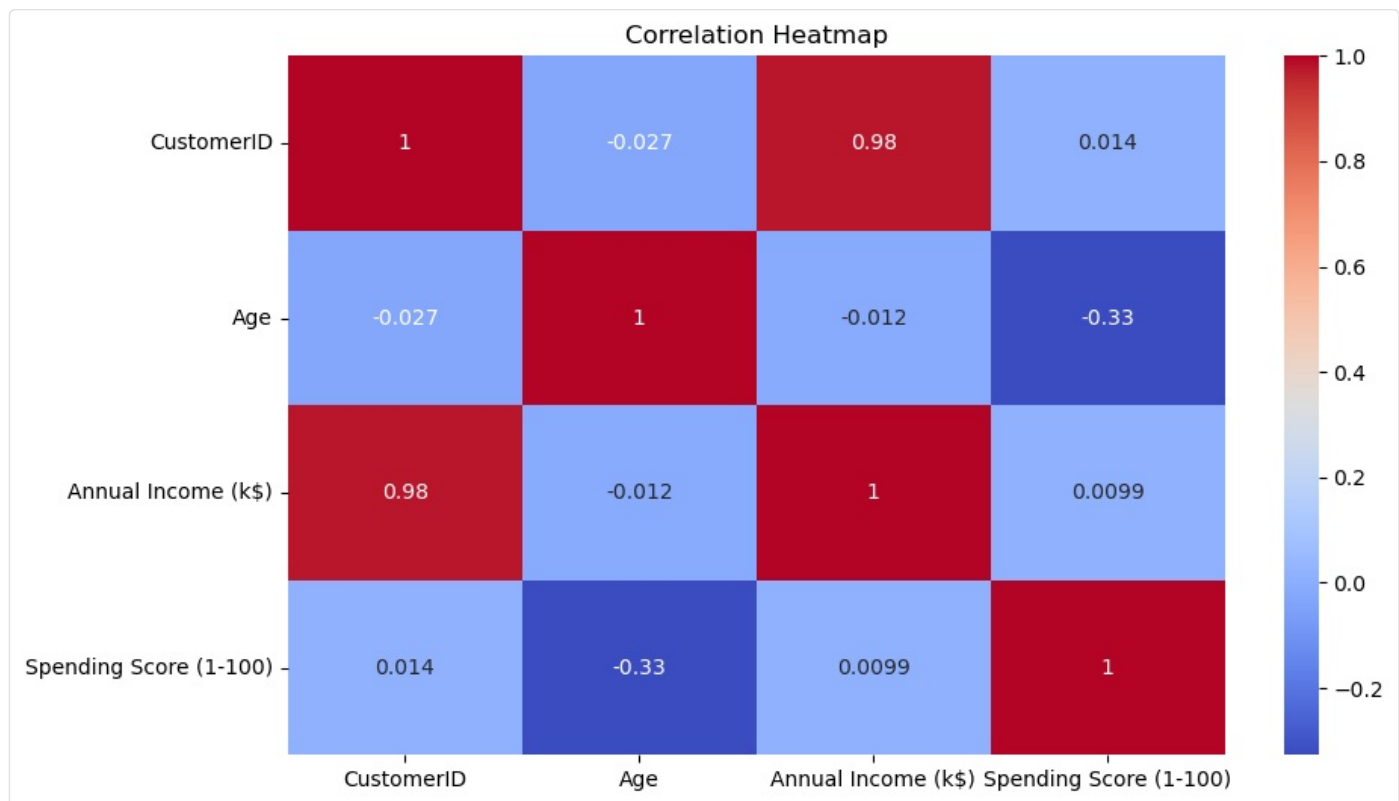


This is the most important chart in the whole EDA. Instead of one blob, the points form five visually distinct clouds: a central cluster of "average" customers, a cluster in the top-right (high income, high spending), a cluster in the bottom-right (high income, low spending), a cluster in the top-left (low income, high spending), and a cluster in the bottom-left (low income, low spending). This is a strong visual signal that K-Means with 5 clusters will find real, meaningful structure here — the segmentation isn't forced.



The pairplot lays out every pairwise combination of Age, Income, and Spending Score, colored by gender. The diagonal (histograms) confirms the univariate shapes above; off-diagonal panels confirm that Age vs Spending Score has a loose downward-sloping cloud (older customers trending toward lower/mid spending, but with a lot of scatter), while Income vs Spending Score shows the same five-cluster pattern seen in the dedicated scatter plot. No strong, clean linear relationship jumps out anywhere except CustomerID vs Income (explained below) — which is exactly why clustering, rather than linear regression, is the right tool here.

4. Correlation heatmap



Reading the heatmap cell by cell:

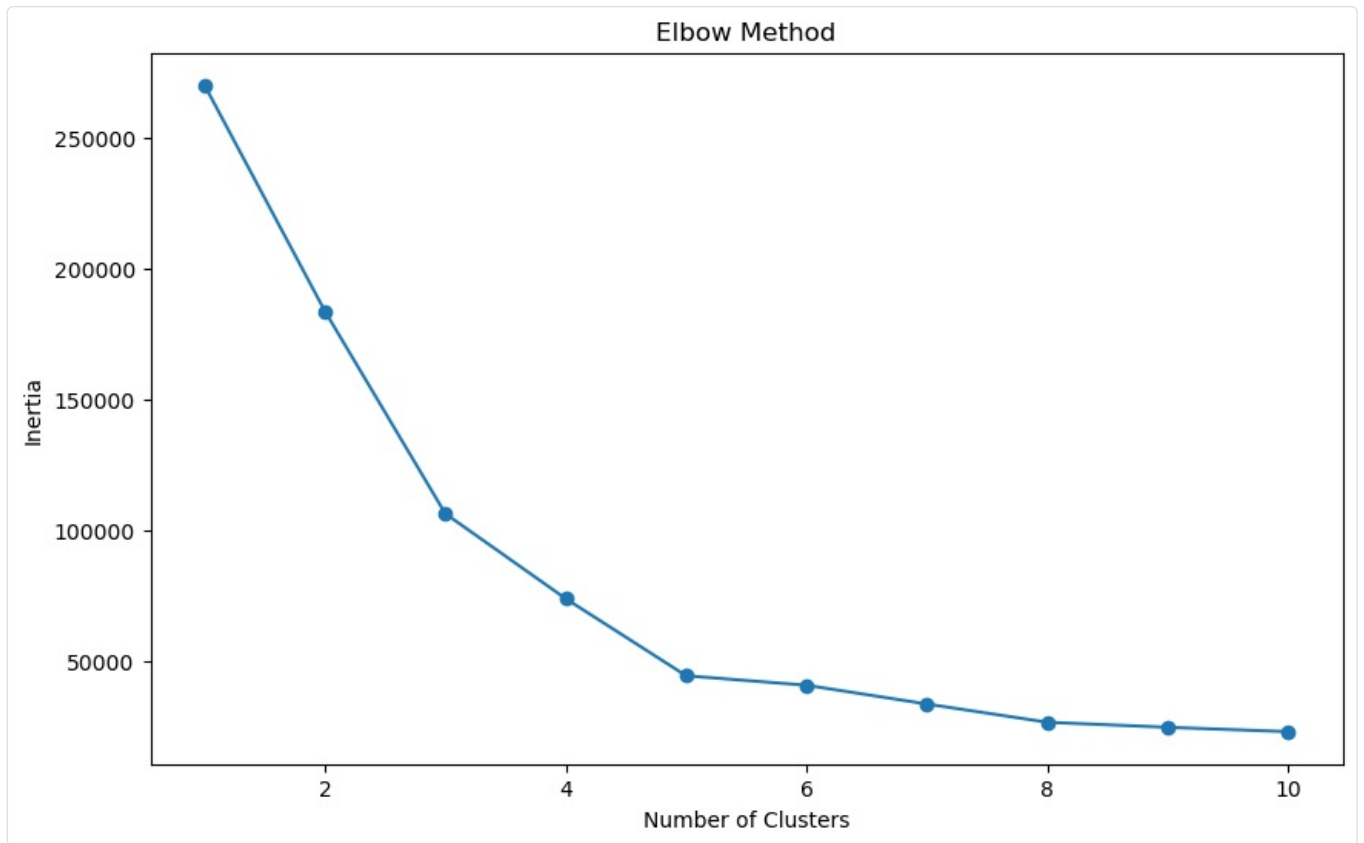
- **CustomerID vs Annual Income (k\$) : 0.98** — a very strong correlation, but it's almost certainly an artifact of how the data was originally sorted/generated (customers appear to be ordered roughly by income) rather than a real business relationship. `CustomerID` is just a row label; this correlation should be ignored/dropped from any modeling, and it was correctly excluded from clustering in the notebook.
- **Age vs Spending Score (1-100) : -0.33** — a weak-to-moderate negative correlation. Older customers tend to have somewhat lower spending scores, but the relationship is far from deterministic (an r of -0.33 explains only about 11% of the variance). This is visible in the Age vs Spending scatter further down: there's a clear cluster of older, low-spending customers, but plenty of older customers spend moderately too.
- **Annual Income (k\$) vs Spending Score (1-100) : 0.0099** — essentially zero linear correlation. This is an important and slightly counter-intuitive finding: **income does not linearly predict spending behavior**. This is precisely why the raw correlation heatmap can be misleading on its own — a near-zero correlation coefficient hides the fact that income and spending score have strong *non-linear, group-based* structure (the five clusters seen in the scatter plot). Correlation only measures straight-line relationships; K-Means is picking up something correlation can't see.
- **Age vs CustomerID / Annual Income (k\$) : both close to 0** — no meaningful relationship, as expected since ID is just an index.

Takeaway: the heatmap tells you *not* to expect a linear regression to work well between income and spending — and that's the justification for using clustering instead.

Clustering Bivariate (Annual Income vs Spending Score)

Features used: `Annual Income (k$)` , `Spending Score (1-100)`

Elbow method

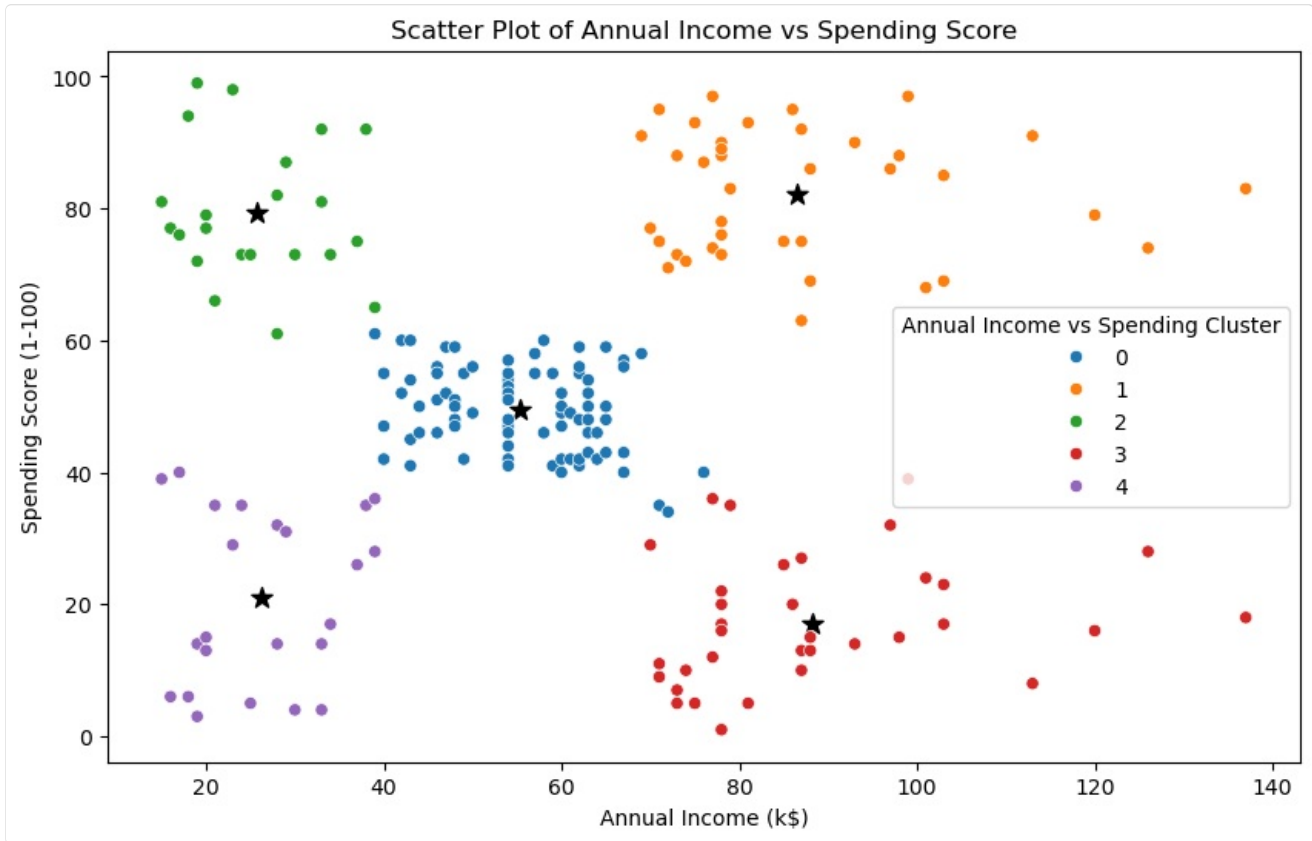


Inertia (within-cluster sum of squared distances) by number of clusters (k):

k	Inertia
1	269,981
2	183,653
3	106,348
4	73,881
5	44,448
6	40,825
7	33,643
8	26,687
9	24,766
10	23,103

How to read this chart: Inertia always decreases as you add more clusters (more clusters = points closer to their own centroid), so the goal isn't the lowest inertia — it's finding the point where adding another cluster stops buying you much improvement. The curve here drops steeply and consistently from k=1 through k=5 (a reduction of over 225,000), then flattens out noticeably from k=5 onward (only ~21,000 total reduction across k=5 through k=10). That bend at **k=5** is the "elbow," and it's why the notebook selects **5 clusters** — it lines up exactly with the five visual clusters spotted in the raw scatter plot during EDA.

Cluster results



Cluster centroids (black stars) and profile of each of the 5 segments (from `groupby` means):

Cluster	Avg Age	Avg Income (k\$)	Avg Spending Score	Segment Label
0	42.7	55.3	49.5	Standard — mid income, mid spending
1	32.7	86.5	82.1	Premium / Target customers — high income, high spending
2	25.3	25.7	79.4	Careless / Impulsive spenders — low income, high spending
3	41.1	88.2	17.1	Careful / Frugal — high income, low spending
4	45.2	26.3	20.9	Sensible / Budget-conscious — low income, low spending

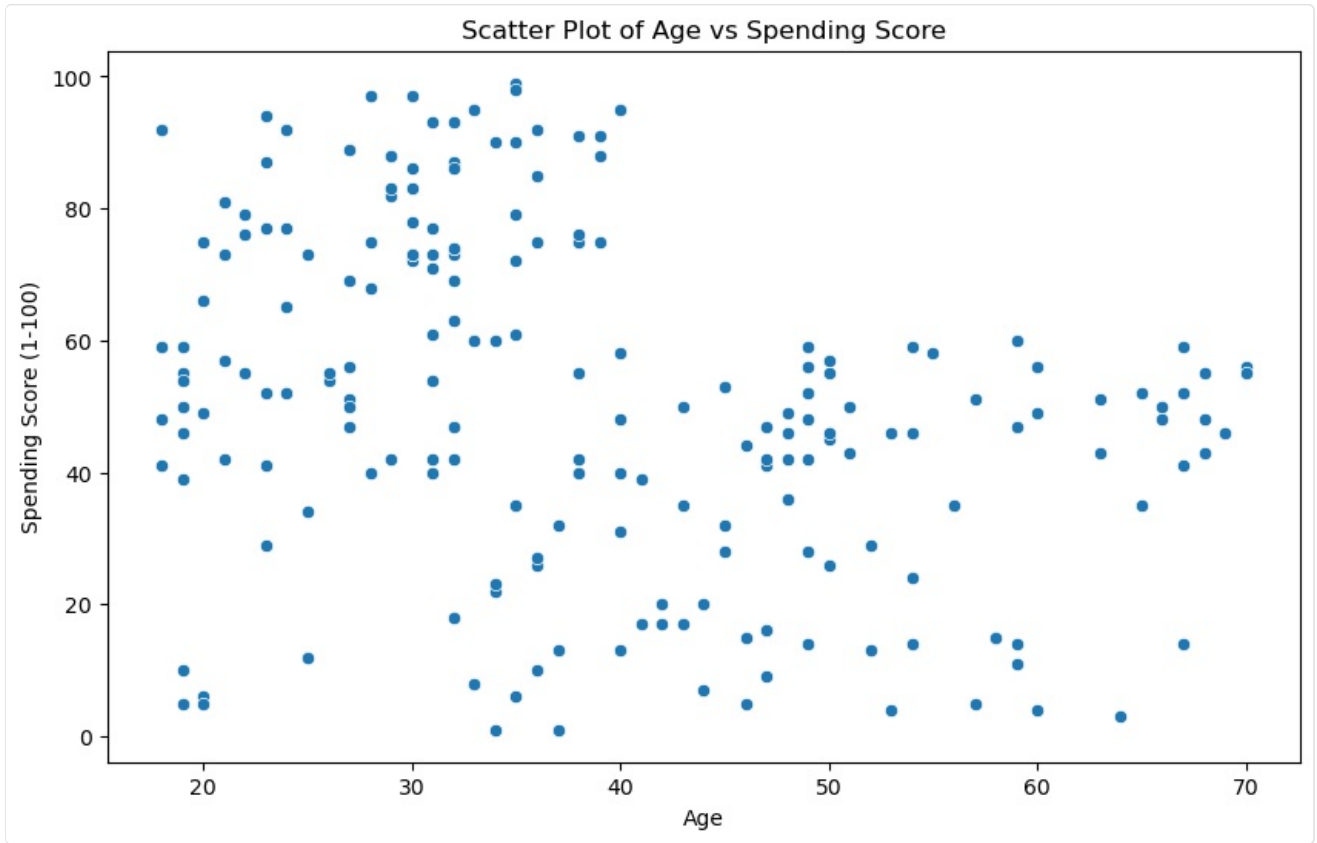
What each cluster means for the business:

- **Cluster 1 (high income, high spending)** is the highest-value segment — these are the customers to prioritize for loyalty programs and premium offers, since they both can and do spend.
- **Cluster 3 (high income, low spending)** is arguably the most interesting group commercially: they have the *purchasing power* but aren't converting it into spend at this mall. This is the natural target for re-engagement campaigns, personalized offers, or investigating why they aren't spending more (competitor pull, product-fit issues, etc.).
- **Cluster 2 (low income, high spending)** — younger customers (avg. age 25) spending beyond what their income alone would predict. Interesting for lower-ticket, high-frequency promotions and impulse-buy merchandising, but with more sensitivity around risk/credit exposure if any credit-based offers are involved.
- **Cluster 4 (low income, low spending)** — the most price-sensitive segment; best served with discount/value messaging rather than premium campaigns.
- **Cluster 0 (mid/mid)** — the "average" customer; general marketing works fine here, no special treatment required.

Sanity-check prediction: the notebook tests the fitted model on a hypothetical customer with income = 80k, spending score = 20, and the model correctly assigns them to **Cluster 3** (high income, low spending) — exactly where a customer with that profile should land, confirming the model is behaving sensibly.

Clustering Bivariate (Age vs Spending Score)

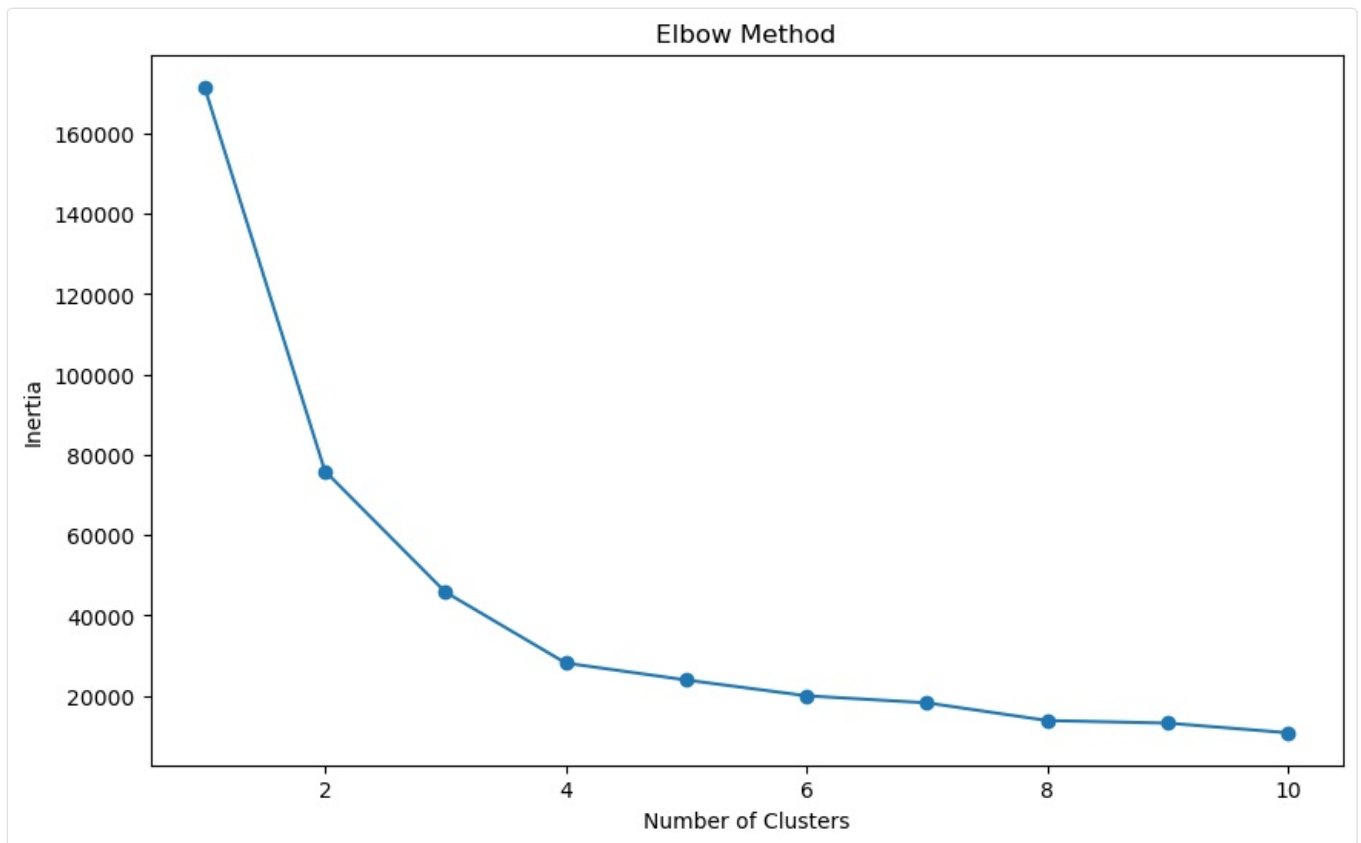
Note: in the original notebook this section's markdown header was accidentally copy-pasted from the first clustering section and still read "Annual Income vs Spending Score." I've relabeled it here to match what the code actually clusters on — **Age vs Spending Score**.



Before clustering, the raw scatter of Age against Spending Score shows a rough triangular/wedge shape: younger customers (roughly under 40) span the full range of spending scores, from very low to very high, while older customers (40+) are increasingly concentrated in the low-to-mid spending range. This matches the -0.33 correlation seen in the heatmap — there's a real but noisy downward trend, and it visually suggests 3–4 natural groupings rather than a single trend line.

Features used: Age , Spending Score (1-100)

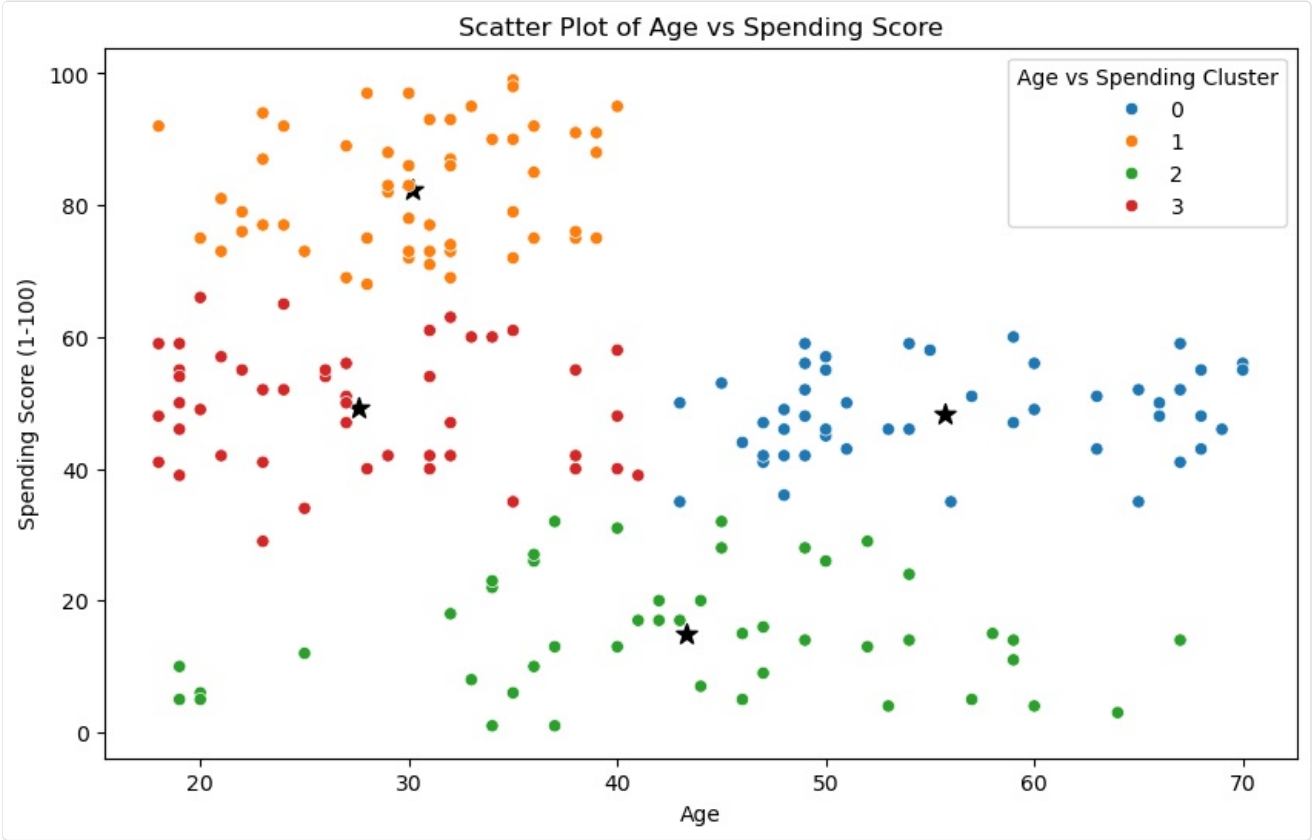
Elbow method



k	Inertia
1	171,536
2	75,949
3	45,841
4	28,166
5	23,951
6	20,016
7	18,257
8	13,856
9	13,216
10	10,808

Reading the curve: the sharpest drops happen from k=1 to k=4 (inertia falls by roughly 143,000 across those four steps), and the marginal improvement from k=4 to k=5 is comparatively small (~4,200) before the curve flattens further. That bend at **k=4** is the elbow, so the notebook fits **4 clusters** for this feature pair — one fewer cluster than the income/spending pairing, consistent with there being a less complex (more linear, less "quadrant-like") relationship between age and spending score.

Cluster results



Cluster centroids (black stars) and profile of each of the 4 segments (computed from the cluster centers):

Cluster	Avg Age	Avg Spending Score	Segment Label
0	55.7	48.2	Older, moderate spenders
1	30.2	82.4	Younger, high spenders
2	43.3	15.0	Middle-aged, low spenders
3	27.6	49.1	Younger, moderate spenders

What each cluster means for the business:

- Cluster 1 (young, high spenders)** — a strong-fit segment for trend-driven, higher-frequency marketing (fashion, lifestyle, experiences) — this age/spend combination is the classic mall power-shopper profile.
- Cluster 2 (middle-aged, low spenders)** — the segment most worth investigating for churn risk or unmet needs; low engagement despite being in a typical high-earning life stage.
- Clusters 0 and 3** are both "moderate spenders" but at different life stages (older vs younger) — worth different messaging even though their spending level is similar, since a 55-year-old and a 28-year-old respond to different channels and offers.

Sanity-check prediction: a hypothetical 60-year-old with a spending score of 50 is predicted into **Cluster 0** (older, moderate spenders)

— consistent with the centroid profile above.

Key Takeaways & Recommendations

- **Income alone doesn't explain spending.** The near-zero linear correlation between income and spending score is the central finding of the EDA — it's the reason clustering (which can find non-linear, group-based structure) outperforms a simple linear model here.
- **The 5-segment income/spending model is the more actionable of the two**, since it separates customers into business-relevant quadrants (premium, frugal-but-wealthy, impulsive, budget-conscious, average) that map directly onto different marketing strategies.
- **The 4-segment age/spending model adds a life-stage lens** on top of that — useful for tailoring *how* you talk to a segment (channel, tone, product type) even when *how much* they spend is similar.
- A few things worth tightening up if you continue developing this project:
- **Feature scaling:** `StandardScaler` is imported but never actually applied before fitting K-Means. Income (roughly 15–137) and Spending Score (1–100) happen to sit on similar scales here, so it doesn't distort results much in this particular dataset, but it's best practice to scale features before any distance-based algorithm — and it becomes essential if you extend to a 3-feature clustering (Age + Income + Spending) since Age has a different scale.
- **Silhouette Score:** `silhouette_score` is imported but never used. Adding it alongside the elbow method gives you a second, more objective way to validate the chosen k (elbow is somewhat subjective/visual; silhouette gives a numeric cluster-quality score between -1 and 1).
- **3-feature clustering:** try clustering on all three numeric features together (Age, Income, Spending Score) to see whether a richer segmentation beats the two separate bivariate models — this is a natural "next step" to mention in a portfolio write-up.
- **Model persistence gap:** the Streamlit app (`app.py`) loads a saved model via `joblib.load('customer_segmentation')` , but the notebook as provided never actually fits and saves that model file (e.g., `joblib.dump(clustering2, 'customer_segmentation')`). Add that save step so the app runs end-to-end for anyone cloning the repo.
- **Repo structure suggestion** for GitHub: `customer-segmentation-clustering/` | — `README.md` | — `Customer_Segmentation_Clustering.ipynb` | — `app.py` | — `customer_segmentation.pkl` # saved model (add `joblib.dump`) | — `data/` | — `Mall_Customers.csv` | — `images/` | — (charts used in this README) Also worth swapping the hardcoded local Windows path (`C:\Users\User\Downloads\...`) for a relative `data/Mall_Customers.csv` path so the notebook runs on any machine, including GitHub Codespaces or Colab.

Tech Stack

Python · pandas · NumPy · scikit-learn (KMeans, StandardScaler, silhouette_score) · seaborn / matplotlib · Streamlit (deployment) · Jupyter Notebook